Named Entity Recognition in French Botanical Text with LSTM Variants

Fahd Chaib · Abdallah Alnajjar · Mohamed Taha El Azizi
Mohamed Houssam El Jabbar · Ismail Ibnouazzi

Advanced Natural Language Processing — Lab Project on Recurrent Neural Networks — 2026/2027
Code: github.com/Fahd-Chaib/floraner-lstm

Summary. We compare a vanilla LSTM with two lighter cells, CIFG (coupled input–forget gate) and LSTMP (LSTM with projection), all implemented from scratch with `nn.Linear`, on named entity recognition in French botanical descriptions (FloraNER). Under an identical protocol, CIFG and LSTMP match the vanilla LSTM (93.3–93.5 % entity F1) with 21–25 % fewer recurrent parameters. Among the hyperparameters studied, the learning rate has the largest effect, mainly on convergence speed.

1. Introduction and goals

We address **named entity recognition (NER)** on FloraNER, a corpus of French botanical descriptions. The model labels each word as a plant organ (*feuilles*, *pétiole*), a descriptor of that organ (*ovales*, *3 cm*), or neither: a sequence labelling task with one label per word. Our goals are (1) to implement a vanilla LSTM baseline without `nn.LSTM`; (2) to implement two non-vanilla cells, CIFG and LSTMP; and (3) to study the effect of the learning rate, optimizer, weight initialization and batch size. Since both variants are smaller than the vanilla cell, our guiding question is: **can a smaller recurrent cell perform as well as the vanilla LSTM on this task?**

2. Dataset and exploratory data analysis

FloraNER contains 838 descriptions with an official split of 670 training and 168 test documents. We use its **coarse-grained** version, with two entity types: ORGANE (a plant part) and DESCRIPTEUR (a word describing it). Table 1 summarizes the data after our preprocessing.

Statistic (coarse-grained)	Value
Documents train / validation / test	603 / 66 / 168
Words per document	mean 254, max 1,277
Segments train / validation / test	10,792 / 1,155 / 3,065
Words per segment	mean 14.2, 95th percentile 38
Share of words labelled O (train)	79.2 %
Entities in train: ORGANE / DESCRIPTEUR	16,040 / 13,158
Vocabulary (min. frequency 2) / unknown test words	3,122 words / 3.1 %

Table 1. FloraNER statistics after preprocessing.

Four observations shaped our design:

- **Character-level annotations.** Entities are given as character offsets. We tokenized the text and converted the spans to the **BIO** scheme (B = beginning, I = inside, O = outside). Only 44 of 32,000 spans did not align with word boundaries (0.1 %). We used the `filtered_annotations` column, which removes the 130 overlapping spans that BIO cannot represent.
- **Long documents.** Descriptions reach 1,277 words, which is slow for a step-by-step recurrent model and weakens gradients over long distances. We split documents into segments at semicolons and double spaces, which separate the descriptions of different organs, without ever cutting inside an entity.

- **Class imbalance.** 79 % of words are O. A model predicting only O would reach 79 % token accuracy while finding no entity, so token accuracy cannot be used for model selection.
- **Noisy labels.** The corpus was annotated automatically (distant supervision) and its authors note that some entities are missing; we observed unlabelled cases such as *colonnaire* or *10-40 m*. This caps the score any model can reach on these labels.

One text appeared in both the training and the test files; we removed it from training to avoid leakage.

3. Modeling approach

All cells are written by hand with `nn.Linear` and plugged into the same tagger, so the cell is the only difference between models.

3.1 Recurrent cells

Vanilla LSTM (eq. 1–6). The input, forget and output gates (i, f, o) and the candidate g update the memory $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ and the state $h_t = o_t \odot \tanh(c_t)$. The four input matrices are stacked in one `nn.Linear(D, 4H)` and the four recurrent ones in one `nn.Linear(H, 4H)`, then split; this is identical to the equations. As a check, we copied the weights of PyTorch's `nn.LSTM` into our cell: the outputs match up to 6×10^{-8}.

CIFG (eq. 13–18). The input gate is tied to the forget gate, so only three weight blocks are learned (−25 % recurrent parameters):

$$i_t = 1 - f_t, \quad c_t = f_t \odot c_{t-1} + (1 - f_t) \odot g_t$$

LSTMP (eq. 19–25). The cell computes $m_t = o_t \odot \tanh(c_t)$ as a vanilla LSTM would, then projects it to a smaller size $P < H$. The recurrent matrices read h_t of size P instead of H; with $H = 128$ and $P = 64$ this saves 21 % of the recurrent parameters:

$$h_t = W_p \, m_t + b_p, \quad W_p \in \mathbb{R}^{P \times H}$$

3.2 Tagger and protocol

The tagger is an **embedding layer** (dimension 100, trained from scratch), a **bidirectional recurrent layer** (one cell reads the segment left to right, a second one right to left, and their states are concatenated), **dropout** 0.3 and a **linear layer** giving one score per BIO tag and per word. Padding is ignored twice: padded positions carry the label −100, skipped by `CrossEntropyLoss`, and a mask freezes (h, c) on padded steps so that the backward direction starts at the last real word. A unit test checks that padding never changes an output.

- **Splits.** Official test file; validation = 10 % of the training *documents* (not segments, to avoid leakage); fixed seed, identical for every model.
- **Vocabulary.** Training set only, lower-cased, digits mapped to 0 (*12 cm* and *35 cm* share one form). Words seen once are mapped to `<unk>`, so the model learns a representation for unknown words.
- **Training.** Cross-entropy loss, gradient clipping at norm 5, at most 30 epochs, early stopping after 5 epochs without improvement of the validation F1; the best checkpoint is evaluated once on the test set. Google Colab, NVIDIA T4 GPU.
- **Metric.** Strict **entity-level F1** (conlleval definition): a predicted entity counts only if its type and both boundaries are exact. We also report precision, recall and the number of trainable parameters.

4. Hyperparameter design choices

Each factor was varied alone on the vanilla model, all others at their default value (H = 128, Adam, learning rate 10^{-3}, PyTorch default initialization, batch 32). Final test F1 stays

between 93.0 % and 93.6 % across all settings (Table 2): hyperparameters change *how fast* the model learns far more than *how well*.

Factor	Setting	Test F1 (%)	Best epoch	s / epoch
Learning rate	10^{-2}	93.54	9	39
	3×10^{-3}	**93.59**	**8**	**39**
	10^{-3} (default)	93.29	9	39
	3×10^{-4}	93.01	23	39
Optimizer	**Adam (default)**	**93.29**	**9**	**39**
	AdamW (weight decay 0.01)	93.26	10	40
	SGD, momentum 0.9, lr 0.1	93.22	23	40
	RMSprop	93.09	15	40
Initialization	Default (uniform)	93.29	9	39
	Xavier	**93.40**	**13**	**39**
	Orthogonal recurrent weights	93.37	13	39
	Orthogonal + forget bias 1	93.36	13	38
Batch size	16	93.43	8	63
	32 (default)	93.29	9	38
	64	93.36	16	23
	128	**93.51**	**19**	**14**

Table 2. One-factor-at-a-time study on the vanilla model (one seed). Best setting of each factor in bold.

- **Learning rate** is the most influential factor. At 3×10^{-4} the model needs 23 epochs and ends lowest; at 3×10^{-3} and 10^{-2} it converges in 8–9 epochs and scores best (Figure 1b).
- **Optimizer.** All four reach about 93 %. Adam converges fastest (9 epochs) and SGD slowest (23 epochs); RMSprop trades precision (90.7 %) for recall (95.6 %).
- **Initialization** has a small effect (+0.1 point for Xavier or orthogonal). A forget-gate bias of 1, which helps on long sequences, changes nothing here: segments are 14 words long, so long-term memory is not the bottleneck.
- **Batch size** barely moves F1 but changes cost: batch 128 is 4.6× faster per epoch than batch 16 (14 s vs 63 s), because each step of the sequential time loop processes more segments at once.

These runs use one seed each. Across three seeds the cells vary by 0.01 to 0.11 point (standard deviation), so differences above 0.2 point are meaningful while differences of 0.1 point or less should be read as ties.

5. Results and evaluation

Table 3 compares the three cells at the same hidden size H = 128, averaged over three seeds, and two enlarged variants with the same recurrent budget as the vanilla cell (one seed).

Model	Recurrent params	Test F1 (%)	Precision (%)	Recall (%)	s / epoch
Vanilla LSTM, H = 128	234,496	93.29 ± 0.01	91.85	94.79	41
CIFG, H = 128	**175,872 (−25 %)**	**93.48 ± 0.08**	**92.11**	**94.89**	**41**
LSTMP, H = 128, P = 64	185,472 (−21 %)	93.35 ± 0.11	92.07	94.67	48
CIFG, H = 154 (matched size)	235,620	93.39	92.49	94.30	41
LSTMP, H = 162, P = 64 (matched size)	234,704	93.42	92.64	94.21	45
LSTMP, H = 128, P = 16	123,936 (−47 %)	93.32	92.23	94.44	45

Table 3. Test results (mean ± std over seeds 42, 1, 2 for the first three rows; one seed otherwise). Recurrent parameters count both directions; total parameters are 547,724 (vanilla), 489,100 (CIFG) and 498,188 (LSTMP).

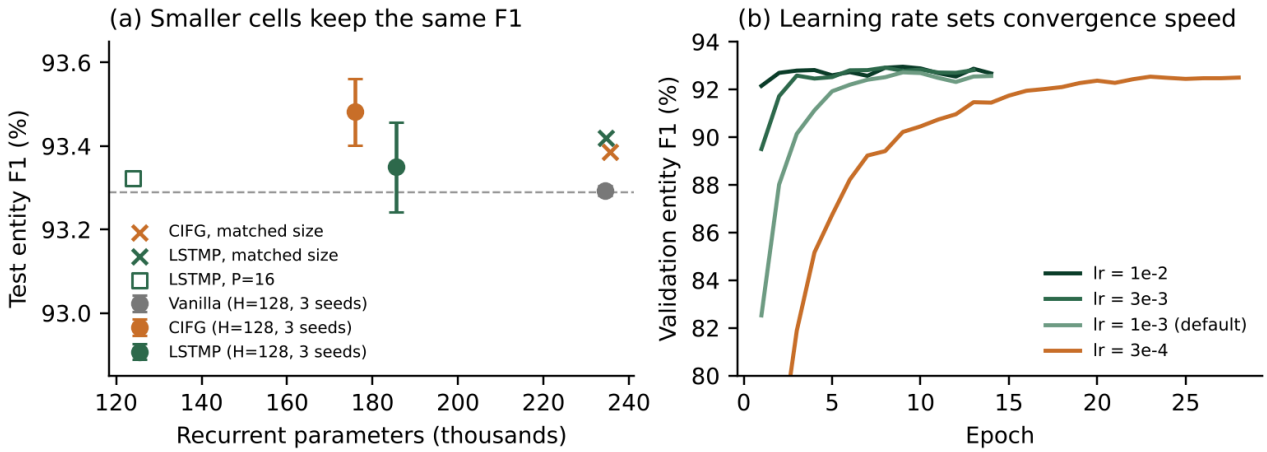


Figure 1. (a) Test entity F1 against recurrent parameters; dashed line = vanilla baseline. (b) Validation entity F1 per epoch for four learning rates (vanilla cell).

- **Smaller cells do not lose accuracy.** CIFG and LSTMP match or slightly exceed the baseline (Figure 1a). The CIFG gain (+0.19 point) is about twice its standard deviation, so it is at most a small effect: the safe conclusion is that the removed parameters were not needed. Shrinking the LSTMP projection to P = 16 still gives 93.32 % with 47 % fewer parameters.
- **More parameters do not help.** Giving CIFG and LSTMP the vanilla budget leaves F1 unchanged (93.4 %): capacity is not the limiting factor on this task.
- **Recall exceeds precision** for every model (≈ 94.7 % vs 92 %): the models predict entities that the annotation lacks, consistent with the missing labels noted in Section 2.
- **Cost.** CIFG trains as fast as vanilla. LSTMP is about 17 % slower per epoch, because the projection adds one matrix product at every time step; its saving is in memory, not time.
- **Per type.** ORGANE is recognized better than DESCRIPTEUR for every model (≈ 95.8 % vs 90.6 % F1). Organs are single words from a closed vocabulary, while descriptors are open-ended and may span several words (*3 cm de longueur*), so one boundary error makes the whole entity wrong. Token accuracy (97.4–97.5 %) hides these differences.

We select **CIFG** as our final model: it has the best average F1, the fewest parameters and the same training time as the baseline. Runs with an identical configuration reproduce the same score exactly (93.29 % for the default vanilla model in five separate runs), so the pipeline is deterministic for a given seed.

6. Discussion, limitations and potential improvements

Why the cells perform alike. Segments average 14 words, so the long-range memory that distinguishes LSTM variants is rarely needed: most decisions depend on the word itself and its immediate neighbours (*feuilles* is an organ; a word after a number is a unit). On such inputs, the extra gate of the vanilla cell and a full-size recurrent state bring no benefit.

Limitations. (i) *Label noise*: some of our “false positives” are probably correct, which lowers measured precision and caps every model. (ii) *Statistical power*: hyperparameters were tested with one seed each. (iii) *One factor at a time*: combinations such as batch 128 with learning rate 3×10^{-3} were not explored. (iv) *Segment context*: splitting at semicolons can separate a descriptor from its organ.

Potential improvements. The plateau around 93.4 % comes from the data rather than the architecture, so the most promising directions act on the input and the labels:

1. **Character-level features** (a character CNN or LSTM per word) to handle rare botanical words through their suffixes (*-ées*, *-iforme*).

2. **Pre-trained French embeddings** (fastText) instead of embeddings learned from 600 documents.
3. **A CRF output layer** to enforce valid BIO sequences and fix boundary errors, the main source of DESCRIPTEUR mistakes.
4. **Label cleaning**: re-annotate a sample to measure label noise, or complete missing labels with confident predictions.
5. **Joint tuning** of learning rate and batch size with a learning-rate schedule and several seeds per setting.

7. Conclusion

We implemented vanilla LSTM, CIFG and LSTMP cells from scratch and compared them on French botanical NER under an identical protocol. Our final model, a bidirectional CIFG tagger, reaches **93.5 % entity F1** on the test set. CIFG and LSTMP match the vanilla LSTM with 25 % and 21 % fewer recurrent parameters, and enlarging them brings no gain. The learning rate is the most influential hyperparameter, and a batch of 128 cuts training time by 4.6× at no cost in accuracy. Further progress depends on richer word representations and cleaner labels rather than on the recurrent cell.

References

1. Nainia A., Vignes Lebbe R., Chenin E., Sahraoui M., Mousannif H., Zahir J. (2024). *FloraNER: a Named Entity Recognition Dataset for Botanical French Text.* Zenodo. doi:10.5281/zenodo.12765869
2. Hochreiter S., Schmidhuber J. (1997). *Long Short-Term Memory.* Neural Computation 9(8).
3. Greff K., Srivastava R. K., Koutník J., Steunebrink B. R., Schmidhuber J. (2017). *LSTM: A Search Space Odyssey.* IEEE TNNLS. arXiv:1503.04069
4. Sak H., Senior A., Beaufays F. (2014). *Long Short-Term Memory Recurrent Neural Network Architectures for Large Scale Acoustic Modeling.* Interspeech.